

DOCUMENTO

Busca Binária

busca-binaria.md

Sumário

Busca Binária	3
1. O problema que ela resolve	3
2. A ideia: o jogo do “maior ou menor”	3
Visualizando	4
3. A implementação básica	4
4. Primeira e última ocorrência	6
Primeira ocorrência (limite esquerdo)	6
Última ocorrência (limite direito)	6
5. As versões prontas da STL	7
6. O custo da ordenação	8
7. Busca binária sobre a resposta	8
8. Erros comuns	9
9. Aplicação: o problema 1520	9
9.1 A preparação, em main	10
9.2 O primeiro laço: existe ou não existe?	10
9.3 A saída antecipada	11
9.4 A divisão do trabalho	12
9.5 O segundo laço: a primeira ocorrência	12
9.6 O terceiro laço: a última ocorrência	13
9.7 O retorno	13
9.8 Por que três laços e não um?	13
Resumo rápido	14

Busca Binária

Este material continua os anteriores de Introdução ao C++ e `vector`. Aqui tratamos do nosso primeiro **algoritmo**: a **busca binária**, que transforma uma busca que levaria milhares de passos em poucas dezenas ou centenas.

1. O problema que ela resolve

Você tem um vetor **ordenado** e quer saber se um valor está lá, e, se estiver, em qual posição.

A forma ingênua é olhar elemento por elemento, chamamos isso de **busca linear**:

```
int buscaLinear (vector<int> &v, int alvo) {  
    for (int i = 0; i < (int) v.size(); i++)  
        if (v[i] == alvo) return i;  
    return -1;  
}
```

cpp

Isso funciona, mas com um vetor de 1.000.000 de elementos, no pior caso você vai precisar fazer 1.000.000 de comparações (imagina que o valor que você procura está na última posição). Se o problema te pedir pra fazer 100.000 consultas, são 10^{11} operações, ou seja, seu programa jamais vai passar no tempo dado (*Time Limit Exceeded*).

A busca binária faz o mesmo trabalho partindo o vetor em dois todas as vezes que faz uma comparação, e continua a procurar em somente um dos lados, no fim isso dá pra gente $\log n$ comparações (no qual n é o tamanho do vetor). Para 1.000.000 de elementos, **20 comparações**. Para 1 bilhão, **30 comparações** somente.

IMPORTANTE

A busca binária **exige que o vetor esteja ordenado**. Sem ordenação, ela devolve resposta errada sem dar nenhum aviso.

2. A ideia: o jogo do “maior ou menor”

Pense na brincadeira em que alguém pensa em um número de 1 a 100 e você tem que adivinhar, recebendo como dica apenas “é maior” ou “é menor”.

Ninguém joga chutando 1, 2, 3, 4... Você chuta **50**. Se a resposta for “maior”, os números de 1 a 50 deixaram de existir: você eliminou metade das possibilidades com um único chute. Depois chuta 75, depois 87, e assim por diante.

A busca binária é exatamente isso:

1. Olhe o elemento do **meio** do intervalo atual.
2. Se ele for o alvo, acabou.
3. Se ele for **menor** que o alvo, o alvo só pode estar à **direita**, então descarte a metade esquerda.
4. Se ele for **maior** que o alvo, o alvo só pode estar à **esquerda**, então descarte a metade direita.
5. Repita até achar ou até o intervalo ficar vazio.

Cada passo corta o problema pela metade. Daí o $\log_2 n$.

Visualizando

Procurando o 7 em $v = \{1, 3, 4, 7, 9, 11, 15\}$ (índices 0 a 6):

```
passo 1:  esq=0      dir=6      meio=3
          [1, 3, 4, (7), 9, 11, 15]
          v[3] == 7  ->  achou no índice 3
```

Procurando o 11:

```
passo 1:  esq=0  dir=6  meio=3  v[3]=7 < 11  -> descarta esquerda, esq = 4
          [1, 3, 4, (7), 9, 11, 15]
                      ^^^^^^^^

passo 2:  esq=4  dir=6  meio=5  v[5]=11 == 11 -> achou no índice 5
          [9, (11), 15]
```

Procurando o 5 (que não existe):

```
passo 1:  esq=0  dir=6  meio=3  v[3]=7 > 5  -> dir = 2
passo 2:  esq=0  dir=2  meio=1  v[1]=3 < 5  -> esq = 2
passo 3:  esq=2  dir=2  meio=2  v[2]=4 < 5  -> esq = 3
passo 4:  esq=3 > dir=2          -> intervalo vazio, não existe
```

3. A implementação básica

```
int buscaBinaria (vector<int> &v, int alvo) {
    int esq = 0; // limite esquerdo do intervalo
    int dir = v.size() - 1; // limite direito do intervalo

    while (esq <= dir) {
        int meio = (esq + dir) / 2;

        if (v[meio] == alvo) return meio; // achou

        if (v[meio] < alvo) esq = meio + 1; // alvo está à direita
        else dir = meio - 1; // alvo está à esquerda
    }

    return -1; // não existe
}
```

cpp

Três detalhes que parecem pequenos e não são:

`esq <= dir`, e não `esq < dir`. Quando `esq == dir`, o intervalo ainda tem **um** elemento para conferir. Trocar por `<` faz o algoritmo ignorar o último candidato e errar em vários casos.

`meio + 1` e `meio - 1`, e não `meio`. Se `v[meio]` não é o alvo, o próprio `meio` já foi descartado. Escrever `esq = meio` faria o intervalo parar de encolher quando sobrassem dois elementos, e o `while` rodaria para sempre.

ATENÇÃO

`(esq + dir) / 2` **pode estourar**. Se `esq` e `dir` forem próximos de 2^{31} , a soma passa do limite do `int`. Uma forma equivalente e segura é a seguinte:

```
int meio = esq + (dir - esq) / 2;
```

cpp

Para vetores (índices pequenos) isso nunca acontece; para busca binária **sobre valores** (seção 7), acontece com frequência.

Por que são equivalentes:

$$esq + \frac{dir - esq}{2} = esq + \frac{dir + esq - 2 \cdot esq}{2} = esq + \frac{esq + dir}{2} - esq = \frac{esq + dir}{2}$$

A conta é a mesma; o que muda é o **caminho** até ela. A primeira forma calcula `esq + dir`, que pode passar do limite do `int`. A segunda nunca soma os dois: ela calcula a **distância** `dir - esq`, que é no máximo o tamanho do vetor, divide por 2 e só então desloca a partir de `esq`. Nenhum valor intermediário chega perto de 2^{31} .

Ou seja, as duas expressões devolvem exatamente o mesmo índice sempre que $0 \leq \text{esq} \leq \text{dir}$, que é justamente a condição do `while`.

ATENÇÃO

`v.size()` devolve um tipo **sem sinal**. Com o vetor vazio, `v.size() - 1` não vira `-1`, vira um número gigantesco, e `dir` recebe lixo. Use `(int) v.size() - 1` quando houver risco de vetor vazio.

4. Primeira e última ocorrência

O código acima devolve **alguma** posição do alvo. Se o vetor tem valores repetidos, qual delas ele devolve depende do formato do vetor; não é previsível.

```
v = {2, 5, 5, 5, 5, 8}
```

cpp

Se o problema pede o **intervalo** ocupado pelo 5 (índices 1 a 4), a busca básica não basta. A solução é não parar quando acha: **registrar** a posição e continuar procurando para o lado que interessa.

Primeira ocorrência (limite esquerdo)

```
int primeiraOcorrencia (vector<int> &v, int alvo) {
    int esq = 0, dir = v.size() - 1, resultado = -1;

    while (esq <= dir) {
        int meio = (esq + dir) / 2;

        if (v[meio] == alvo) {
            resultado = meio;           // guarda este candidato
            dir = meio - 1;             // mas continua procurando à ESQUERDA
        }
        else if (v[meio] < alvo) esq = meio + 1;
        else dir = meio - 1;
    }

    return resultado;
}
```

cpp

A única mudança em relação ao básico é: ao encontrar o alvo, em vez de `return`, o código anota a posição e **força a busca para a esquerda**. Se houver uma ocorrência mais à esquerda, ela sobrescreverá `resultado`. Quando o laço acabar, `resultado` guarda a menor posição encontrada.

Última ocorrência (limite direito)

Idêntico, com a direção invertida:

```
int ultimaOcorrencia (vector<int> &v, int alvo) {
    int esq = 0, dir = v.size() - 1, resultado = -1;

    while (esq <= dir) {
        int meio = (esq + dir) / 2;

        if (v[meio] == alvo) {
            resultado = meio;          // guarda este candidato
            esq = meio + 1;            // mas continua procurando à DIREITA
        }
        else if (v[meio] < alvo) esq = meio + 1;
        else dir = meio - 1;
    }

    return resultado;
}
```

cpp

NOTA

Note que ambas continuam sendo $O(\log n)$. Guardar o resultado e seguir não aumenta o número de passos: o intervalo continua sendo cortado pela metade a cada iteração.

Com as duas, a contagem de ocorrências sai de graça:

```
int quantidade = ultimaOcorrencia(v, x) - primeiraOcorrencia(v, x) + 1;
```

cpp

5. As versões prontas da STL

O C++ já traz busca binária pronta. Todas exigem vetor ordenado e todas são $O(\log n)$.

Função	O que devolve
<code>binary_search(v.begin(), v.end(), x)</code>	true ou false: o valor existe?
<code>lower_bound(v.begin(), v.end(), x)</code>	iterador para a primeira posição com valor $\geq x$
<code>upper_bound(v.begin(), v.end(), x)</code>	iterador para a primeira posição com valor $> x$
<code>equal_range(v.begin(), v.end(), x)</code>	um pair com os dois iteradores acima

Para transformar o iterador em índice, subtraia `v.begin()`:

```
vector<int> v = {2, 5, 5, 5, 5, 8};

int primeiro = lower_bound(v.begin(), v.end(), 5) - v.begin(); // 1
int depois   = upper_bound(v.begin(), v.end(), 5) - v.begin(); // 5
int ultimo   = depois - 1;                                     // 4
int quantos  = depois - primeiro;                             // 4
```

cpp

Cuidado com o caso “não existe”: `lower_bound` **não** avisa. Ela devolve onde o valor *deveria* estar. É preciso conferir:

```
auto it = lower_bound(v.begin(), v.end(), x);

if (it == v.end() || *it != x) cout << "nao existe\n";
else cout << "esta no indice " << (it - v.begin()) << "\n";
```

cpp

DICA

Em competição, prefira `lower_bound` / `upper_bound`: são menores, testadas e não têm risco de laço infinito. Escrever a busca à mão vale a pena quando a condição de parada é mais complicada que “igual a x” (é o caso da seção 7) ou quando o objetivo é justamente treinar o algoritmo.

6. O custo da ordenação

A busca binária é $O(\log n)$, mas ordenar é $O(n \log n)$. Se você vai fazer **uma** busca, ordenar não compensa: a busca linear, em $O(n)$, é mais rápida.

A busca binária compensa quando há **muitas consultas sobre o mesmo vetor**: ordena-se uma vez, $O(n \log n)$, e cada uma das q consultas custa $O(\log n)$, totalizando $O((n + q) \log n)$, contra $O(n \cdot q)$ da linear.

7. Busca binária sobre a resposta

Uma aplicação menos óbvia, e muito comum em olimpíadas: nem sempre se busca em um vetor. Sempre que a resposta for um número dentro de um intervalo conhecido e existir uma função que diga “esse valor serve?” de forma **monótona** (se um valor serve, todos os maiores servem; ou o contrário), dá para buscar binariamente na própria resposta.


```
int esq = 1, dir = 1000000000, resposta = -1;

while (esq <= dir) {
    int meio = esq + (dir - esq) / 2;

    if (serve(meio)) {           // funciona? então tente um valor menor
        resposta = meio;
        dir = meio - 1;
    }
    else esq = meio + 1;         // não funciona, precisa de valor maior
}
```

cpp

É o mesmo esqueleto da seção 4: guardar o candidato e continuar para o lado que pode melhorar a resposta.

8. Erros comuns

Erro	O que acontece
Buscar em vetor não ordenado	resposta errada, sem aviso nenhum
Usar <code>while (esq < dir)</code>	o último elemento nunca é conferido
Usar <code>esq = meio</code> em vez de <code>meio + 1</code>	laço infinito quando sobram 2 elementos
<code>(esq + dir) / 2</code> com valores enormes	<i>overflow</i> do <code>int</code> ; use <code>esq + (dir - esq) / 2</code>
<code>dir = v.size() - 1</code> com vetor vazio	<code>dir</code> vira um número gigante (tipo sem sinal)
Parar no primeiro <code>==</code> quando há repetidos	devolve uma posição qualquer, não a primeira
Usar <code>lower_bound</code> sem checar <code>*it != x</code>	trata “não existe” como se existisse
Esquecer de limpar o vetor entre casos de teste	o caso seguinte busca em dados do anterior

9. Aplicação: o problema 1520

O problema 1520 traz um enunciado em que se recebem **lotes de parafusos**, cada lote descrito por um intervalo `inicio..fim` de números. Todos esses números são jogados em um único vetor e, dado um `alvo`, é preciso responder em **quais posições** ele aparece (do primeiro ao último índice), ou que ele não existe.

Ou seja: não basta achar o alvo, é preciso achar **as duas pontas** do bloco de repetições. É exatamente o cenário da seção 4.

9.1 A preparação, em `main`

```
while (cin >> lotes) {
    for (int i = 0; i < lotes; i++) {
        cin >> inicio >> fim;

        for (int j = inicio; j <= fim; j++) {
            parafusos.push_back(j);
        }
    }
    cin >> alvo;

    sort(parafusos.begin(), parafusos.end());
}
```

cpp

Duas coisas acontecem aqui que são **pré-requisito** da busca binária:

- **Os lotes são expandidos** em números individuais com `push_back`. Cada lote vira uma sequência crescente, mas lotes diferentes podem se sobrepor, por isso o mesmo valor pode aparecer várias vezes no vetor final.
- **`sort` é obrigatório**. Cada lote isolado já vem crescente, mas a concatenação de vários lotes não é. Sem essa linha, a busca binária daria resposta errada. É também o `sort` que agrupa as cópias repetidas do alvo em um **bloco contíguo**; sem isso, “do índice X ao índice Y” nem faria sentido.

```
    parafusos.clear();
}
```

cpp

O `clear()` no fim do laço é o que permite tratar vários casos de teste com o mesmo vetor. Sem ele, o caso seguinte buscaria também nos parafusos do anterior (o penúltimo erro da tabela da seção 8).

9.2 O primeiro laço: existe ou não existe?

```
int esq = 0, esq1, esq2;
int dir = parafusos.size() - 1, dir1, dir2;
int meio = -1, meio1, meio2;
int resultado, resultado1 = -1, resultado2 = -1;

// encontrando o primeiro elemento com o valor do alvo
while (esq <= dir) {
    meio = (esq + dir) / 2;

    if (parafusos[meio] == alvo) {
        resultado1 = meio;
        resultado2 = meio;
        break;
    }

    if (parafusos[meio] < alvo) esq = meio + 1;
    else dir = meio - 1;
}
```

cpp

Este é o algoritmo da seção 3, sem nenhuma modificação. Ele tem **duas** funções aqui:

1. **Decidir se o alvo existe.** Se sair do laço sem `break`, o alvo não está no vetor.
2. **Achar um ponto de apoio.** O `meio` em que ele parou é *alguma* posição do alvo, não necessariamente a primeira nem a última, mas garantidamente dentro do bloco de repetições.

O `break` é essencial: ele preserva os valores de `esq`, `dir` e `meio` no instante da descoberta. Esses três valores serão reaproveitados a seguir.

Note que `resultado1` e `resultado2` já recebem `meio`. Isso cobre o caso em que o alvo aparece **uma única vez**: os laços seguintes não encontrarão nada melhor, e a resposta será `{meio, meio}`, que é a correta.

9.3 A saída antecipada

```
if (resultado1 == -1 && resultado2 == -1) {
    return {resultado1, resultado2};
}
```

cpp

Se o alvo não foi encontrado, ambos continuam `-1` e a função devolve `{-1, -1}` imediatamente. Isso evita rodar os dois laços seguintes à toa e, mais importante, evita usar `meio`, que nesse caso guarda um índice qualquer, sem significado.

É esse `{-1, -1}` que o `main` testa para imprimir `not found`:

```
if (intervalo.first == -1 && intervalo.second == -1) {
    cout << alvo << " not found\n";
}
```

9.4 A divisão do trabalho

```
// definindo as variáveis
esq1 = esq; dir1 = meio - 1;
esq2 = meio + 1; dir2 = dir;
```

cpp

Esta é a linha mais inteligente do código, e a que exige mais atenção.

O primeiro laço já eliminou tudo que estava fora de `[esq, dir]`, sabe-se que o alvo não pode estar lá. E dentro desse intervalo, a posição `meio` já é conhecida. Sobram então **duas faixas ainda não examinadas**:

```

      esq                meio                dir
      |                  |                  |
[ ... | ---- faixa 1 ---- (X) ---- faixa 2 ---- | ... ]
      esq1 .. dir1                esq2 .. dir2
```

- **Faixa 1**, de `esq` até `meio - 1`: onde pode haver ocorrências **mais à esquerda** que `meio`.
- **Faixa 2**, de `meio + 1` até `dir`: onde pode haver ocorrências **mais à direita** que `meio`.

Em vez de reiniciar cada busca do zero (0 até `size() - 1`), o código aproveita o trabalho já feito. É uma otimização: as duas buscas seguintes rodam sobre intervalos já reduzidos.

9.5 O segundo laço: a primeira ocorrência

```
while (esq1 <= dir1) {
    meio1 = (esq1 + dir1) / 2;
    if (parafusos[meio1] == alvo) {
        resultado1 = meio1;
        dir1 = meio1 - 1;
    }
    else if (parafusos[meio1] < alvo) esq1 = meio1 + 1;
    else dir1 = meio1 - 1;
}
```

cpp

É a `primeiraOcorrencia` da seção 4, aplicada à faixa 1. O comportamento importante está no primeiro `if`: ao encontrar o alvo, **não** para. Grava `resultado1` e faz `dir1 = meio1 - 1`, empurrando a busca para a esquerda.

Se existir uma ocorrência ainda anterior, ela será encontrada e sobrescreverá `resultado1`. O último valor gravado é, por construção, o **menor índice** em que o alvo aparece.

E se a faixa 1 não contiver nenhuma ocorrência? O laço termina sem alterar nada, e `resultado1` mantém o valor de `meio` gravado no primeiro laço, que continua sendo a resposta certa, pois nesse caso `meio` é a primeira ocorrência.

9.6 O terceiro laço: a última ocorrência

```
while (esq2 <= dir2) {
    meio2 = (esq2 + dir2) / 2;
    if (parafusos[meio2] == alvo) {
        resultado2 = meio2;
        esq2 = meio2 + 1;
    }
    else if (parafusos[meio2] < alvo) esq2 = meio2 + 1;
    else dir2 = meio2 - 1;
}
```

cpp

Espelho exato do anterior, sobre a faixa 2. A diferença é apenas a direção: ao achar o alvo, faz `esq2 = meio2 + 1`, empurrando a busca para a **direita**, para tentar um índice maior. `resultado2` termina com o **maior índice** em que o alvo aparece.

9.7 O retorno

```
return {resultado1, resultado2};
```

cpp

A função devolve um `pair<int, int>`, a forma mais direta de retornar dois valores de uma vez. No `main`:

```
pair<int, int> intervalo = buscaBinaria(parafusos, alvo);
...
cout << alvo << " found from " << intervalo.first << " to " << intervalo.second << "\n";
```

cpp

`intervalo.first` é a primeira ocorrência e `intervalo.second` é a última.

9.8 Por que três laços e não um?

Seria possível rodar `primeiraOcorrencia` e `ultimaOcorrencia` diretamente, sem o laço inicial, seriam duas buscas em vez de três. A solução do problema 1520 opta por:

1. uma busca para **decidir a existência** (e sair cedo se não existir);
2. duas buscas em intervalos **já reduzidos** pela primeira.

As duas abordagens são $O(\log n)$; a diferença é constante. A vantagem desta é tratar `not found` num único ponto, sem repetir a checagem nos dois lados.

NOTA

Todo o miolo da função (os três laços) cabe em quatro linhas com a STL: `cpp auto ini = lower_bound(parafusos.begin(), parafusos.end(), alvo); auto fim = upper_bound(parafusos.begin(), parafusos.end(), alvo); if (ini == fim) return {-1, -1}; // não existe return {ini - parafusos.begin(), fim - parafusos.begin() - 1};` Escrever à mão, como na solução do problema 1520, é o que mostra que o algoritmo foi entendido, mas em prova vale saber que o atalho existe.

Resumo rápido

```
sort(v.begin(), v.end()); // SEMPRE antes de buscar

// busca simples
while (esq <= dir) {
    int meio = esq + (dir - esq) / 2;
    if (v[meio] == alvo) return meio;
    if (v[meio] < alvo) esq = meio + 1;
    else dir = meio - 1;
}
return -1;

// primeira ocorrência: ao achar -> resultado = meio; dir = meio - 1;
// última ocorrência: ao achar -> resultado = meio; esq = meio + 1;

// versões prontas
binary_search(v.begin(), v.end(), x); // existe?
lower_bound(v.begin(), v.end(), x); // primeiro >= x
upper_bound(v.begin(), v.end(), x); // primeiro > x
```

cpp